

CONSIDER THE FOLLOWING
SITUATION

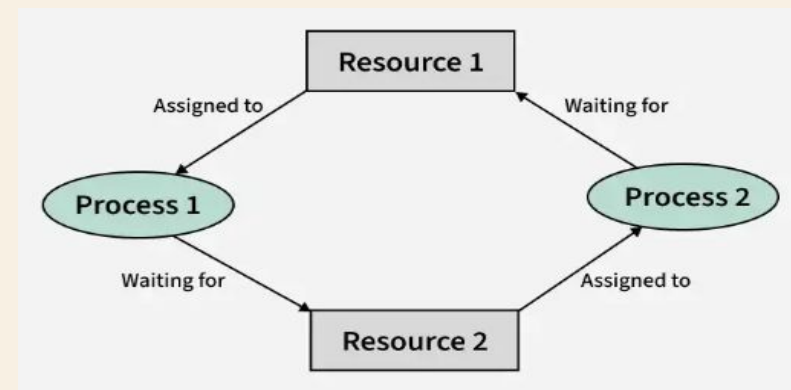
RESULT



DEADLOCK

DEADLOCK

Real Life	Operating System
Husband	Process/Thread
Wife	Process/Thread
Fixing light	Resource
Making tea	Resource



HOW DEADLOCK OCCURS

Multiple processes compete for limited resources

Each process holds at least one resource

Each process requests additional resources

Requested resources are held by other waiting processes

Result: No process can proceed

SYSTEM MODEL (DEADLOCK CONTEXT)

RESOURCE USAGE CYCLE (NORMALLY)

1. Request

- The thread requests a resource
- If unavailable → thread enters waiting state

2. Use

- Once allocated, the thread uses the resource
- Performs its task (e.g., executes critical section)

3. Release

- After completion, the thread releases the resource
- Resource becomes available for others

DEADLOCK AWARENESS IN MULTITHREADING

Developers must consider the possibility of deadlocks in multithreaded systems

Synchronization tools like mutexes and semaphores are used to prevent race conditions

Improper use of these tools can still lead to deadlock

Correct handling of lock acquisition and release is critical

Key Idea: Preventing race conditions does not guarantee avoiding deadlocks

SAMPLE C CODE

- A mutex is a resource in this context.
- Thread 1 locks: **first** → **second**
- Thread 2 locks: **second** → **first**
- This inconsistent order can cause deadlock:
 - Thread 1 holds first_mutex, waits for second_mutex
 - Thread 2 holds second_mutex, waits for first_mutex
- Both wait forever → Deadlock occurs

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t first_mutex, second_mutex;

void *do_work_one(void *param){
    pthread_mutex_lock(&first_mutex);sleep(1);
    pthread_mutex_lock(&second_mutex);
    printf("Thread1 working\n");
    pthread_mutex_unlock(&second_mutex);
    pthread_mutex_unlock(&first_mutex);
    pthread_exit(0);}

void *do_work_two(void *param){
    pthread_mutex_lock(&second_mutex);sleep(1);
    pthread_mutex_lock(&first_mutex);
    printf("Thread2 working\n");
    pthread_mutex_unlock(&first_mutex);
    pthread_mutex_unlock(&second_mutex);
    pthread_exit(0);}

int main(){
    pthread_t t1,t2;
    pthread_mutex_init(&first_mutex,NULL);
    pthread_mutex_init(&second_mutex,NULL);
    pthread_create(&t1,NULL,do_work_one,NULL);
    pthread_create(&t2,NULL,do_work_two,NULL);
    pthread_join(t1,NULL);
    pthread_join(t2,NULL);
    pthread_mutex_destroy(&first_mutex);
    pthread_mutex_destroy(&second_mutex);
    return 0;}
```

DEADLOCK AND SCHEDULING BEHAVIOR

Deadlock is possible but not guaranteed

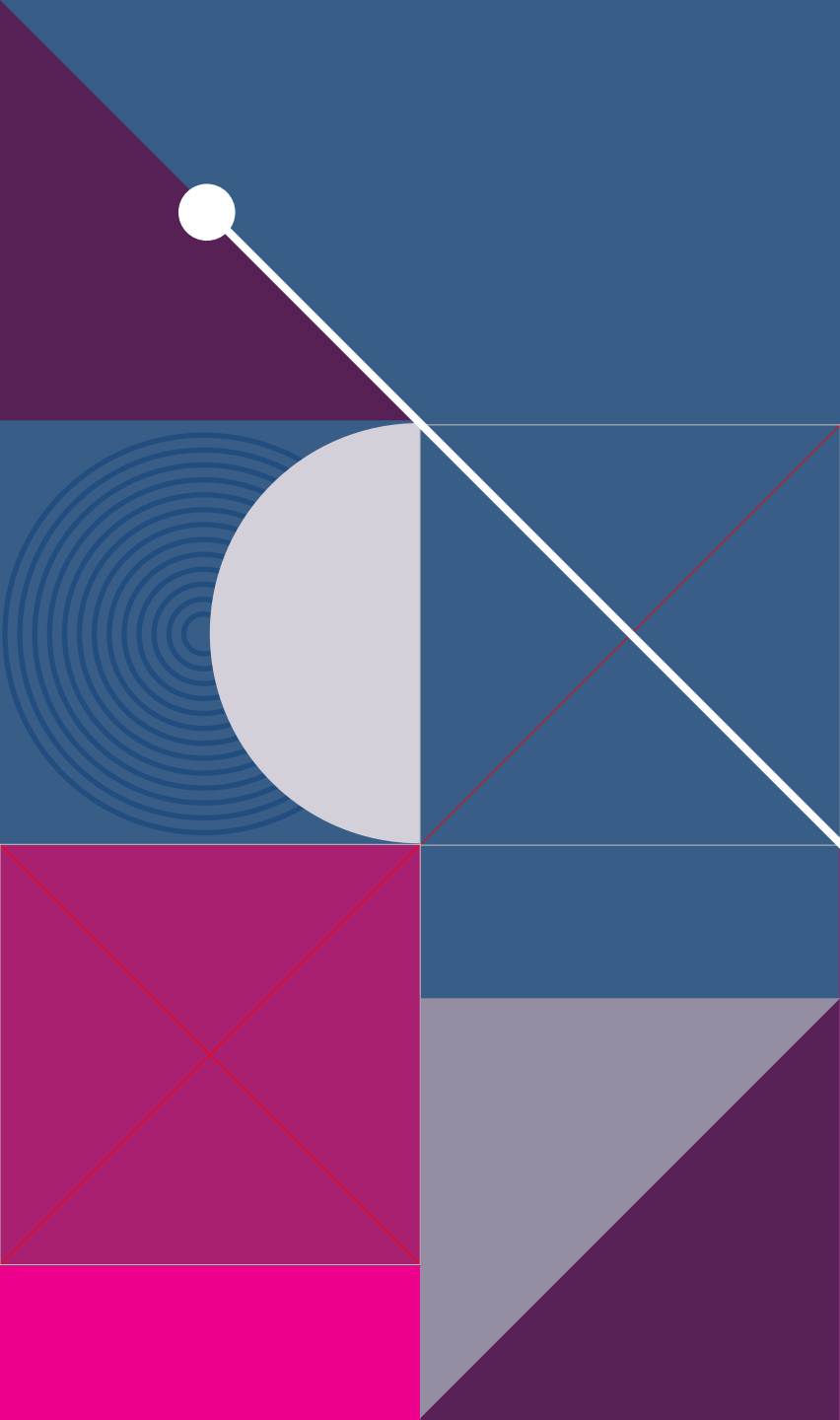
It will not occur if one thread completes its lock sequence before the other starts

Deadlock depends on thread execution order

Execution order is controlled by the CPU scheduler

Different schedules can lead to different outcomes

Deadlocks are hard to detect and test because they occur only under specific timing and scheduling conditions



DEADLOCK CHARACTERIZATION

NECESSARY CONDITIONS FOR DEADLOCK

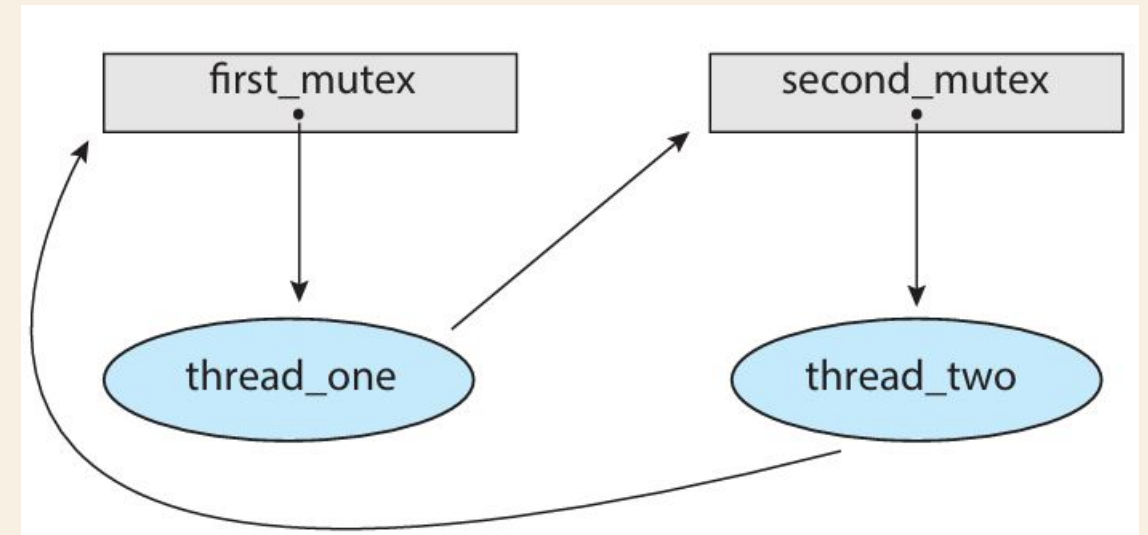
Mutual Exclusion : At least one resource is non-shareable (only one thread can use it at a time)

Hold and Wait : A thread holds at least one resource and is waiting for additional resources

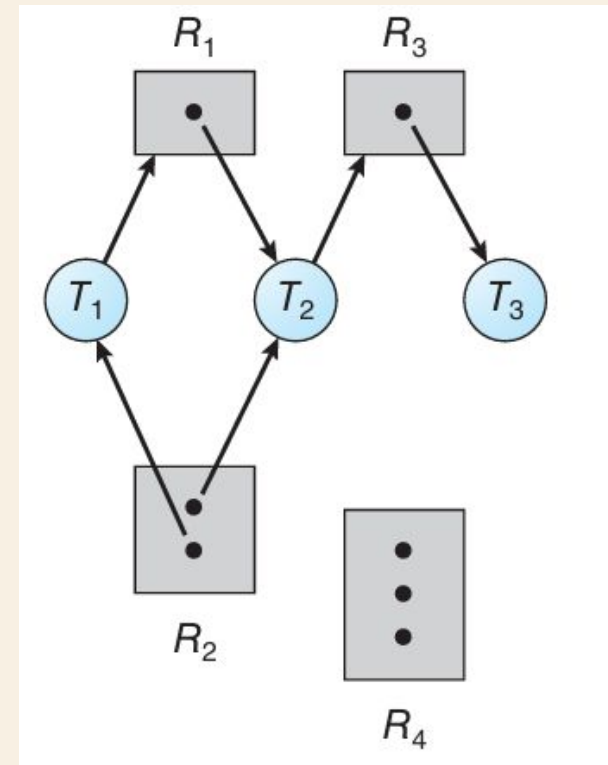
No Preemption : Resources cannot be forcibly taken; they must be released voluntarily

Circular Wait : A circular chain exists where each thread is waiting for a resource held by another

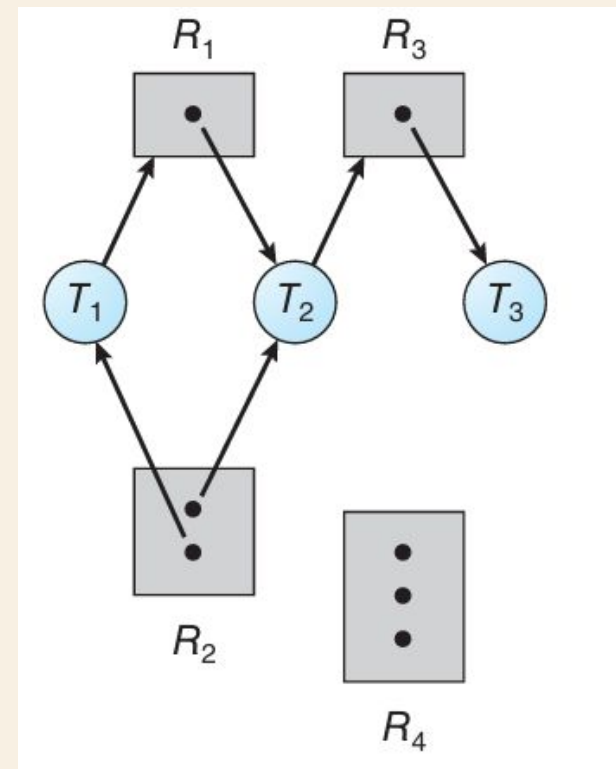
RESOURCE-ALLOCATION GRAPH



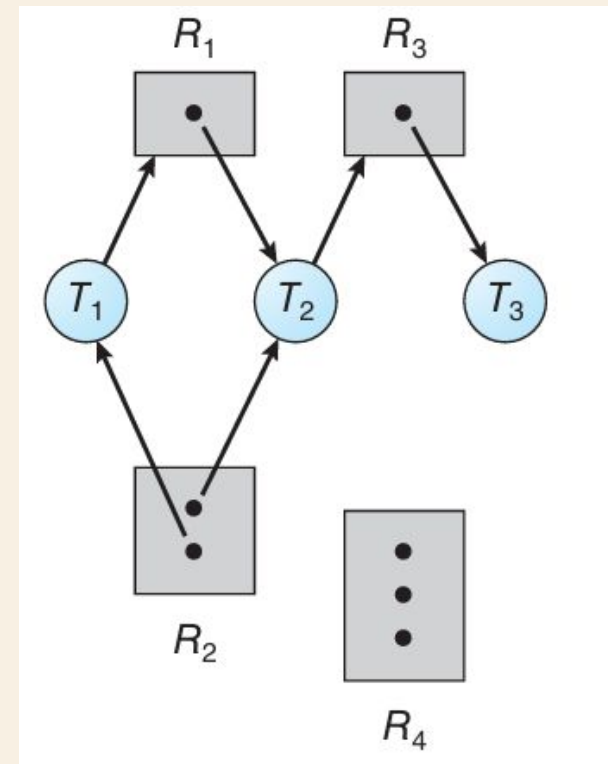
RESOURCE-ALLOCATION GRAPH NOTATION



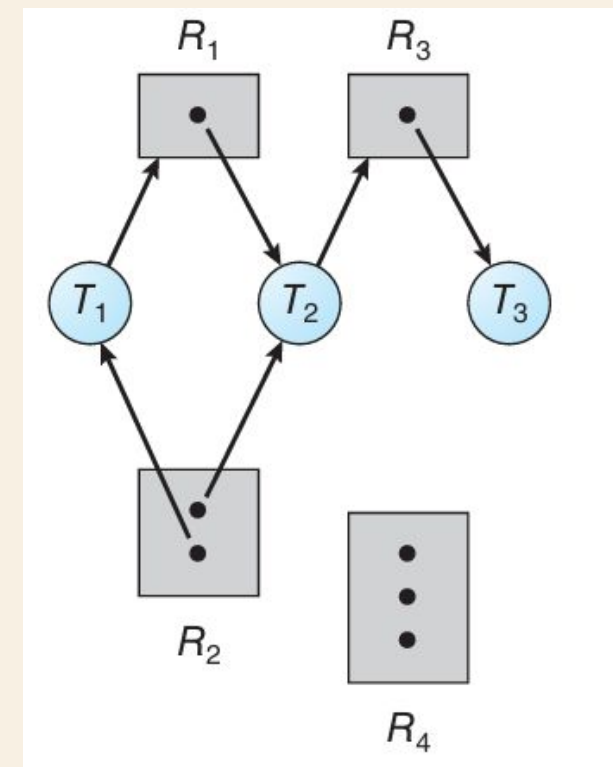
EXAMPLE



EXAMPLE



EXAMPLE



DEADLOCK AND RESOURCE-ALLOCATION GRAPH (KEY CONDITIONS)

If the graph has no cycle, then no deadlock exists

If the graph contains a cycle,
then:

A deadlock may exist

CASE 1: SINGLE INSTANCE PER RESOURCE TYPE

Each resource type has only one instance

A cycle $\Rightarrow$ deadlock is guaranteed

Cycle is both:

- Necessary condition
- Sufficient condition

CASE 2: MULTIPLE INSTANCES PER RESOURCE TYPE

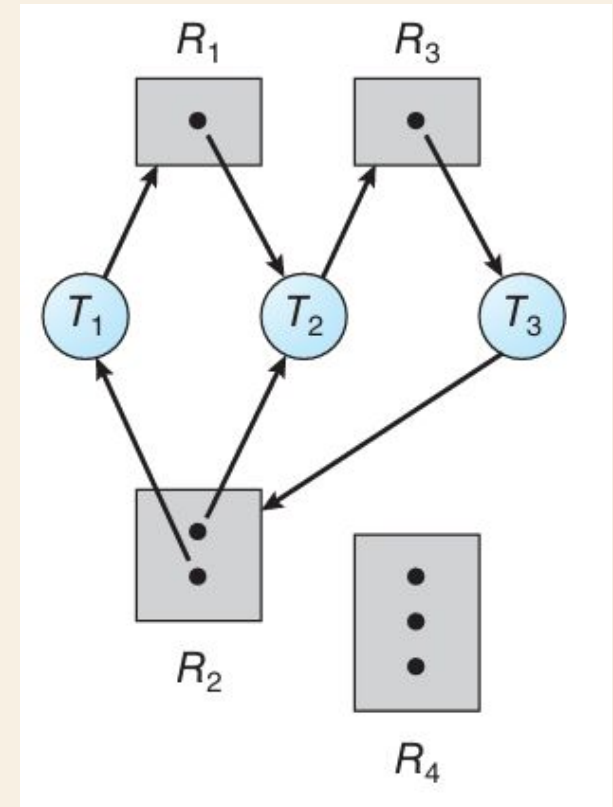
Each resource type has several instances

A cycle $\Rightarrow$ deadlock may exist (not guaranteed)

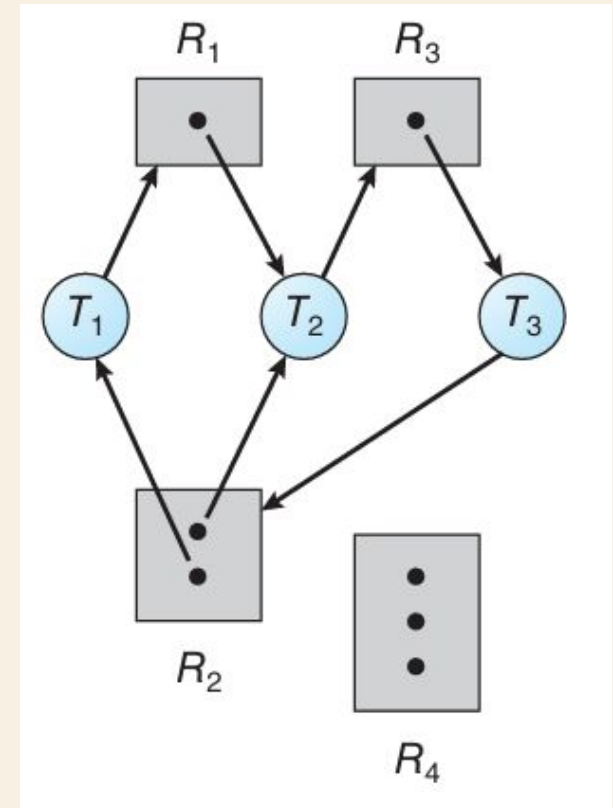
Cycle is only:

- Necessary condition
- Not sufficient condition

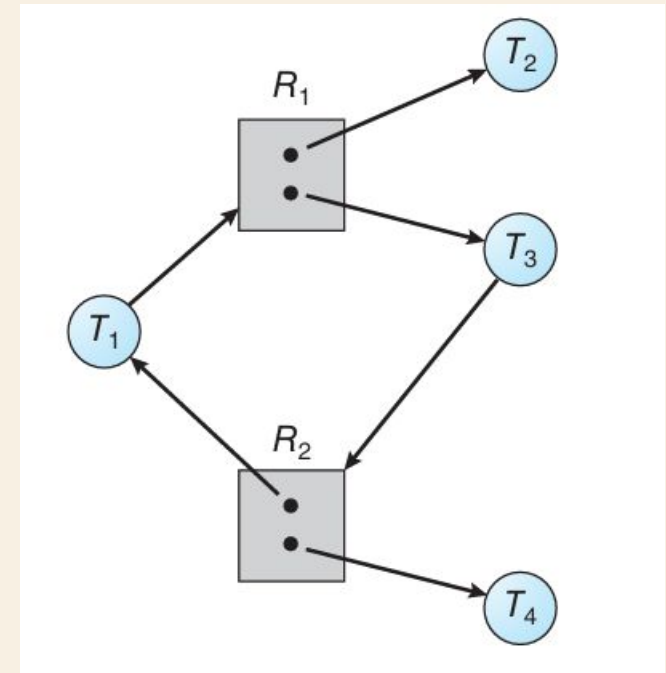
ANOTHER SCENARIO



HERE IS A DEADLOCK



ANOTHER EXAMPLE



METHODS FOR HANDLING DEADLOCKS

Ignore Deadlocks (Ostrich Approach)

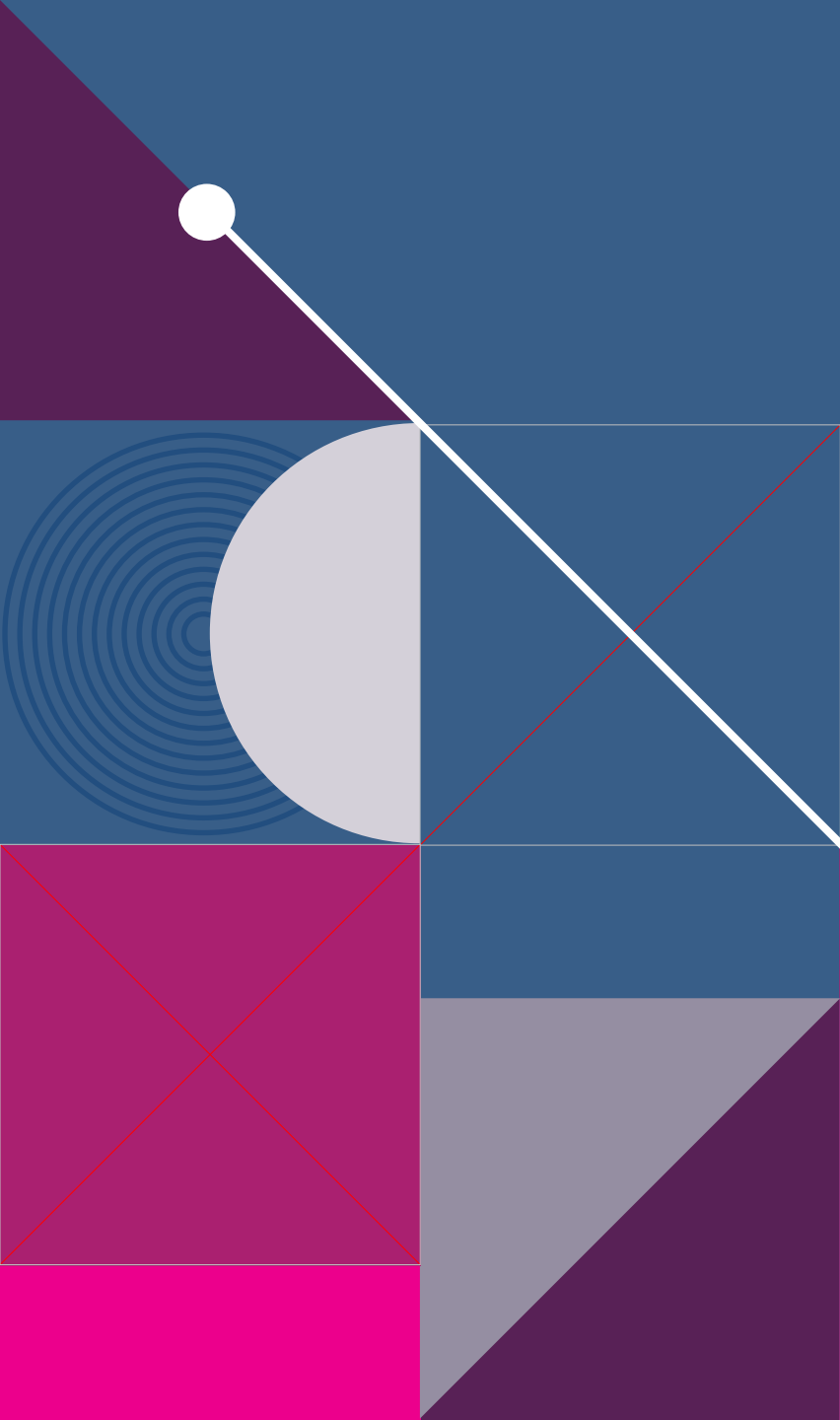
- Assume deadlocks never occur (Mostly used)
- No special handling mechanism

Prevention/Avoidance

- Use protocols to prevent or avoid deadlocks
- Ensure the system never enters deadlock state

Detection and Recovery

- Allow deadlocks to occur
- Detect them and take actions to recover



DEADLOCK PREVENTION

DEADLOCK PREVENTION

We can stop deadlock before they happen.

Deadlock Prevention: at least one necessary condition for deadlock never holds

Result: Deadlock becomes impossible

STRATEGY

Deadlock occurs only if all 4 conditions are true

Prevention works by breaking any one condition

No complete set $\rightarrow$ No deadlock

CONDITIONS TO BREAK

Mutual Exclusion

Hold and Wait

No Preemption

Circular Wait

Prevent any one → avoid deadlock

DEADLOCK PREVENTION (BREAKING CONDITIONS)

Mutual Exclusion:

- Make resources shareable where possible
- Example: use read-only resources

Hold and Wait

- Require threads to request all resources at once
- Or release held resources before requesting new ones

No Preemption

- Allow system to forcibly take resources from a thread
- Reassign resources to other waiting threads

Circular Wait

- Impose a total ordering of resources
- Threads must request resources in a fixed (increasing) order

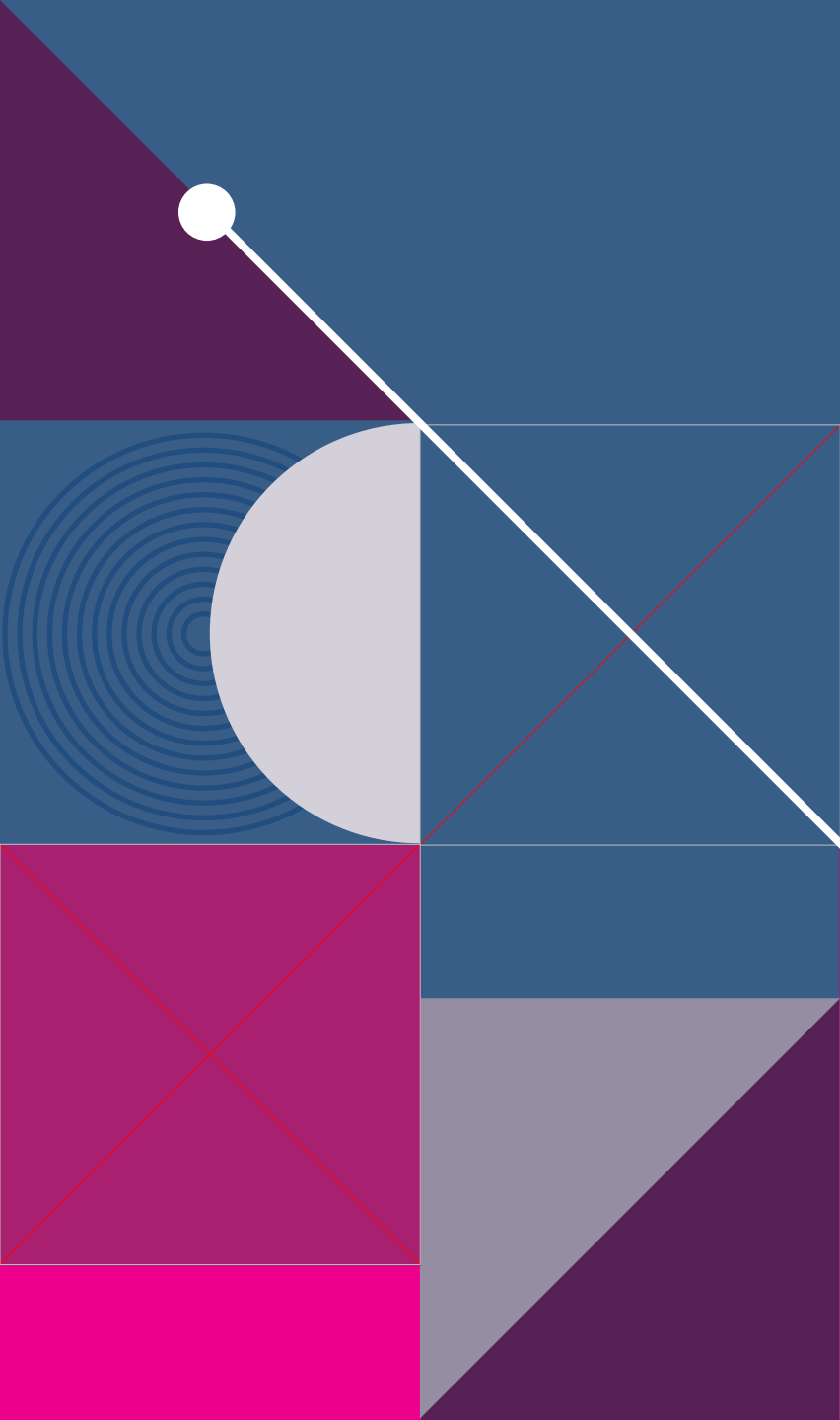
TRADE-OFFS OF DEADLOCK PREVENTION CONDITIONS

Mutual Exclusion: Not always avoidable; some resources must remain exclusive

Hold and Wait: Requires requesting all resources at once → low resource utilization

No Preemption: Forcing release of resources can lead to inconsistent states or overhead

Circular Wait: Imposing ordering on resources reduces flexibility → complex resource management



DEADLOCK AVOIDANCE

BASIC IDEA

Requires prior knowledge of resource requests

Each thread specifies its maximum resource needs

System decides dynamically whether to grant a request

Decision based on:

- Available resources
- Allocated resources
- Future demands of threads

Ensures system stays in a safe state (no circular wait possible)

SAFE STATE

A system is in a safe state if it can allocate resources to all threads without causing deadlock

There exists at least one safe sequence of execution

Safe Sequence: An order of threads where each can obtain its maximum required resources, execute, and release them

Key Idea: If a safe sequence exists → Deadlock can be avoided

SAFE VS UNSAFE STATE

Safe State:

At least one safe sequence exists
Deadlock can be avoided

Unsafe State:

No safe sequence exists
May lead to deadlock

Deadlock State:

System is already stuck

BANKER'S ALGORITHM

A deadlock avoidance algorithm that ensures the system stays in a safe state

Each process declares its maximum resource requirement in advance

On each request, the system:

- Pretends to allocate resources
- Checks if a safe sequence exists

If safe → grant request

If unsafe → deny request (process waits)

EXAMPLE : FINDING SAFE SEQUENCE IF EXISTS

Threads: T0, T1, T2, T3, T4

Resource Types: A, B, C

Total Resources:

- A = 10 instances
- B = 5 instances
- C = 7 instances

The snapshot shows:

- Allocation (currently assigned resources)
- Max (maximum demand of each thread)
- Available (remaining resources in system)

This data is used to determine whether the system is in a safe state or not.

Available = Total Resources – Allocation

Available = A:10,B:5,C:7 – A:7,B:2,C:5

Available = A:3 , B:3, C:2

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

EXAMPLE : CALCULATING AVAILABLE

Available = Total Resources – Allocation
Available = A:10,B:5,C:7 – A:7,B:2,C:5
Available = A:3 , B:3, C:2

Thread	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
T0	0	1	0	7	5	3	3	3	2
T1	2	0	0	3	2	2			
T2	3	0	2	9	0	2			
T3	2	1	1	2	2	2			
T4	0	0	2	4	3	3			
Total	7	2	5						

EXAMPLE : CALCULATING NEED

$$\text{Need} = \text{Max} - \text{Allocation}$$

Thread	Allocation			Max			Available			Need		
	A	B	C	A	B	C	A	B	C	A	B	C
T0	0	1	0	7	5	3	3	3	2	7	4	3
T1	2	0	0	3	2	2				1	2	2
T2	3	0	2	9	0	2				6	0	0
T3	2	1	1	2	2	2				0	1	1
T4	0	0	2	4	3	3				4	3	1

EXAMPLE : T0'S NEED CAN'T BE FULFILLED, SO CHOOSE T1.

Once T1's need is completed, so it will release its allocated resources.
Now the updated available resources are A:5, B: 3 and C:2

Thread	Allocation			Max			Available			Need		
	A	B	C	A	B	C	A	B	C	A	B	C
T0	0	1	0	7	5	3	3	3	2	7	4	3
T1	2	0	0	3	2	2	5	3	2	1	2	2
T2	3	0	2	9	0	2				6	0	0
T3	2	1	1	2	2	2				0	1	1
T4	0	0	2	4	3	3				4	3	1

Safe Sequence : T1

EXAMPLE : NOW T2'S NEED CAN'T BE FULFILLED, SO CHOOSE T3.

Once T3's need is completed, so it will release its allocated resources.
Now the updated available resources are A:7, B: 4 and C:3

Thread	Allocation			Max			Available			Need		
	A	B	C	A	B	C	A	B	C	A	B	C
T0	0	1	0	7	5	3	3	3	2	7	4	3
T1	2	0	0	3	2	2	5	3	2	1	2	2
T2	3	0	2	9	0	2	7	4	3	6	0	0
T3	2	1	1	2	2	2				0	1	1
T4	0	0	2	4	3	3				4	3	1

Safe Sequence : T1, T3

EXAMPLE : T4 CAN BE ALLOCATED EASILY SO CHOOSE T4

Once T4's need is completed, so it will release its allocated resources.
Now the updated available resources are A:7, B: 4 and C:5

Thread	Allocation			Max			Available			Need		
	A	B	C	A	B	C	A	B	C	A	B	C
T0	0	1	0	7	5	3	3	3	2	7	4	3
T1	2	0	0	3	2	2	5	3	2	1	2	2
T2	3	0	2	9	0	2	7	4	3	6	0	0
T3	2	1	1	2	2	2	7	4	5	0	1	1
T4	0	0	2	4	3	3				4	3	1

Safe Sequence : T1, T3, T4

EXAMPLE : FINALLY, T0 OR T2 BOTH CAN BE ALLOCATED EASILY SO CHOOSE ANY

So let's say we choose T0 first.

Thread	Allocation			Max			Available			Need		
	A	B	C	A	B	C	A	B	C	A	B	C
T0	0	1	0	7	5	3	3	3	2	7	4	3
T1	2	0	0	3	2	2	5	3	2	1	2	2
T2	3	0	2	9	0	2	7	4	3	6	0	0
T3	2	1	1	2	2	2	7	4	5	0	1	1
T4	0	0	2	4	3	3				4	3	1

Safe Sequence : T1, T3, T4, T0, T2

UNSAFE STATE

If need of the threads can't be fulfilled it is unsafe state.
In other words, we there is no safe sequence.

Thread	Available			Need		
	A	B	C	A	B	C
T0	3	3	2	7	4	3
T1				4	2	2
T2				6	0	0
T3				0	1	8
T4				4	3	1